

Prague University of Economics and Business

Faculty of Informatics and Statistics



WHEAT YIELD PREDICTION USING REGRESSION APPROACH

BACHELOR THESIS

Study programme: Applied Informatics

Specialization: Applied Informatics

Author: Vojtěch Kuzdas

Supervisor: Ing. MUDr. Lubomír Štěpánek

Prague, May 2022

Acknowledgment

In the first place, my sincere thanks go to Ing. MUDr. Lubomír Štěpánek. His guidance and considerable experience proved invaluable while writing this thesis and therefore I am very grateful for his supervision. In the second place, I would like to express my appreciation to Ing. Josef Horálek Csc. for the timeliness of formal and grammatical editing.

I also wish to express my appreciation and gratitude to Ing. Jiří Pinkas and Ing. Jaroslav Bačina for data provision and Ing. Martin Nedbal for agriculture expert advice.

Abstract

This bachelor thesis explores the applicability of linear and tree regression models in predicting the true yield of wheat crops in the South Moravian and Ústí nad Labem regions in the Czech Republic.

Thesis pairs Sentinel-2 satellite imagery with single-season post-harvest yield data into a 10x10m grid. Yield data are split into percentile and manually filtered datasets. Two vegetation indices, NDVI and EVI, are extracted from satellite imagery and used as the primary independent variables in model training. Some models use additional post-harvest variables as independent variables. Simple and multiple linear regression and tree regression with bagging are done to predict the true yield. All models across all datasets are compared using standard R^2 and MSE statistics. Models based on heavily filtered datasets achieve R^2 values in the upper range of 0.92. Models based on less filtered datasets achieve R^2 values of 0.65. However, the industrial applicability of constructed models requires further discussion due to potential issues (e.g. overfitting). Tree-based models showed better performance over linear regression models. Although perceived promisingly, tree-bagging did not prove to be effective on the given data.

Keywords

Yield Prediction, Remote Sensing, Precision Agriculture, Linear Regression, Regression Trees, Bagging

Content

Introduction	6
1 Technologies allowing data acquisition	8
1.1 Remote sensing and vegetation indices	8
1.2 Sentinel 2 mission	9
1.3 Yield monitoring methods	9
2 Methods and materials	11
2.1 Study area	11
2.1.1 South Moravian Region, Znojmo district	11
2.1.2 Ústí nad Labem Region, Litoměřice district	12
2.2 Yield data acquisition (dependent variable acquisition)	12
2.3 Image data acquisition (independent variable acquisition)	15
2.4 Preprocessing	15
2.4.1 Synchronizing coordinate reference system	15
2.4.2 Manual filtering	15
2.4.3 Percentile filtering	16
2.4.4 Concave hull masking	17
2.4.5 TIN interpolation	17
2.4.6 Raster alignment	18
2.4.7 Extracting vegetation indices	18
2.4.8 Exporting	18
2.4.9 Note on programmatic preprocessing with R language	21
2.5 Regression analysis	21
2.5.1 Linear regression	21
2.5.2 Prediction using simple linear regression	24
2.5.3 Prediction using multiple linear regression	24
2.5.4 Regression trees	24
2.5.5 Prediction using single-variable regression trees	29
2.5.6 Prediction using multi-variable regression trees	30
3 Results and Discussion	31
3.1 Filtering	31
3.2 Image acquisition	31
3.3 Linear regression models	31

3.4 Regression tree models	33
3.5 Best models	35
Conclusions	37
List of references	38
Annexes	I
Annex A: Source code samples	I

Introduction

For 2.5 million years humans fed themselves by gathering plants and hunting animals that lived and bred without their intervention. All this changed about 10,000 years ago, when Sapiens began to devote almost all their time and effort to manipulating the lives of a few animal and plant species. From sunrise to sunset humans sowed seeds, watered plants and plucked weeds from the ground and led sheep to prime pastures. This work, they thought, would provide them with more fruit, grain and meat. It was a revolution in the way humans lived – the Agricultural Revolution.

- Yuval Noah Harari, *Sapiens*

With the ever more increasing world's population, expected to be almost at 10 million by 2050, rises the demand for agricultural products as well. However, supply capabilities have been decreasing due to reduced land availability and climate change (FAO, 2017). Many solutions have been proposed, ranging from dietary prescription changes (Stehfest et al., 2009) to better farm management practices (Lal, 2004). This thesis explores the latter, namely Precision Agriculture (PA).

In the literature, PA is defined as a management strategy that processes field data in combination with other information to support decisions for improved resource use efficiency, productivity and profitability of agricultural production (McBratney et al., 2005).

Field data exploited by PA methods may come in many forms: multi-temporal imagery, crop yield, crop chlorophyll content, moisture and nitrogen levels, pH and others. Moreover, with the recent invention of the Global Positioning System (GPS) PA methods can even combine the precise position with the above-mentioned data allowing for site-specific management – a farmer may, for example, be able to target nitrogen-deficient area of arable land and provide it with special treatment (Chlingaryan et al., 2018; Goel et al., 2003; Rodrigues et al., 2018).

Yield prediction (YP), a promising subset of PA, might be of particular interest to the farm decision-maker. YP methods can determine the site-specific crop yield given the correct data set even before the crop is harvested. These methods usually involve two datasets, one of them containing vegetation index (VI) and the other one containing actual crop yield. Several authors have already described such an approach (Canata et al., 2021; Mehdaoui and Anane, 2020; Panda et al., 2010; Wahab et al., 2018). VIs are obtained using remote sensing methods; recently released free Sentinel 2 imagery (satellite images), or even images provided by unmanned aerial vehicles (drones) (Feng et al., 2020). Crop yield can be obtained using a harvester or, as demonstrated by Mehdaoui and Anane

(2020), grains from randomly placed squares can be counted in a laboratory. However, the latter approach, which may seem more laborious, eliminates many limitations of the harvester obtained data since such data contain a variety of inherent errors. Depending on the grain flow meter system used and its calibration, harvester errors have a form of positional errors, rapid velocity changes, grain through harvester lag and overlapping errors to name a few (Sudduth et al., 2012). These two datasets are then processed into pixels, such that each pixel from one dataset corresponds to a pixel with the same coordinates from the other dataset. Assuming the values of the vegetation indices correlate with the crop yield, using both datasets synchronized in this manner, we can train or design a predictive model.

The main goal of this thesis is to explore the applicability of linear regression and regression tree models in predicting the true wheat crop yield before the harvest itself. The models will be given only a vegetation index extracted from satellite images as the independent variable. Secondly, I will try to improve the models by training them with additional post-harvest variables. Standardized statistics will evaluate the resulting models, and the best model will be chosen. The thesis also concerns data acquisition and processing for such predictive tasks and the pitfalls that may arise.

1 Technologies allowing data acquisition

1.1 Remote sensing and vegetation indices

Equation Chapter (Next) Section 1 Remote sensing (RS) is a set of methods used to make assessments about Earth's surface without making physical contact with it (*ex situ*). In essence, RS methods measure radiant energy (mostly sunlight) reflected by the Earth's surface. The intensity of sunlight reflected is dependent on the coverage of the surface from which it is reflected. Moreover, different types of surfaces absorb or reflect specific wavelengths of electromagnetic radiation differently. For example; healthy plants absorb a wavelength of 625-740 nm (red spectral region) due to chlorophyll and reflect the wavelength of 780-2500 nm (near-infrared spectral region). Given these and alike assumptions, RS authors frequently construct a vegetation index (VI) which helps to display certain characteristics of a given surface area.

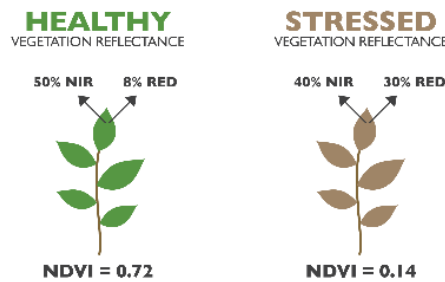


Figure 1.1: Illustration of crop stress level on its NDVI, source: <https://www.agricolus.com/en/vegetation-indices-ndvi-ndmi/>

Huete et al. (2002) define VI as a spectral transformation of multiple bands designed to amplify specific vegetation characteristics and their interseasonal, structural and biophysical variation. They, for example, define a chlorophyll sensitive VI, the normalized difference vegetation index as:

$$NDVI = \frac{NIR - RED}{NIR + RED}, \quad (1.1)$$

where NIR denotes near-infrared reflectance and RED denotes red reflectance. NDVI values range from -1 to 1, indicating moisture-stressed vegetation and a higher density of green vegetation respectively (Gessesse and Melesse, 2019). The higher the NVDI index, the healthier the plant is. See Figure 1.1 above for an illustrative explanation.

Enhanced Vegetation Index (EVI) is another VI improving the NDVI performance in high biomass regions. This VI tries to limit the influence of canopy background signal and the atmospheric influence. EVI is defined by the equation

$$EVI = G \frac{NIR - RED}{NIR + c_1 - c_2 \times BLUE + L}, \quad (1.2)$$

where, according to Huete et al. (2002) ¹:

- G is the gain factor with the value of 2.5
- c_1 and c_2 indicate aerosol resistance, default value is 6 and 6.5 respectively
- L is the canopy background adjustment factor equal to 1

Thus, the formula (1.2) can be written as:

$$EVI = 2.5 \frac{NIR - RED}{NIR + 6 \times RED - 6.5 \times BLUE + 1}, \quad (1.3)$$

where NIR and RED are the same coefficients as in the NDVI algorithm above and BLUE denotes the blue band reflectance coefficient.

1.2 Sentinel 2 mission

The 23rd of June 2015 marks the launch of Sentinel-2A, the first satellite of the European Union's Earth observation mission Copernicus Sentinel-2. The first launch was followed by a second soon after; Sentinel-2B launched in 2017, completing the start of the mission comprising a constellation of two polar-orbiting satellites, phased at 180° to each other. Sentinel-2 mission allows for higher image resolution (Sentinel's agriculture dedicated bands have 10 m resolution whereas Landsat 8, an alternative open-access satellite, for example, only has 30m) and more frequent site revisit due to its two satellite constellation (Sentinel has revisit time of 10 days whereas Landsat has 16 days) (Segarra et al., 2020).

1.3 Yield monitoring methods

Yield monitors are a relatively recent development in agricultural machinery that allow grain producers to assess the effects of weather, soil properties and management on grain production. They are the logical first step for those who want to begin practicing PA (Shearer et al., 1999).

¹ It is important to note, that the study was done with MODIS system which tracks reflectance in value range between 0 and 1. Reflectance values obtained by other systems, such as Sentinel 2, might need to be normalized or mapped onto the same range in order to use algorithm mentioned in the study.

Yield monitors are a combination of several components. They are relatively complex devices, typically including several sensors and components. The sensors measure the mass of grain flow, separator speed, ground speed, grain moisture, and header width. Yield is determined as a product of the various parameters being sensed. The essential component of a yield monitoring system is, however, the sensor used to measure the mass of grain moving through the separator of a combine (Shearer et al., 1999).

The most common approach, as well as the approach employed in this thesis, is mass-flow sensing. This approach is accomplished by assessing the impact force of grain hitting a plate. The paddles of the clean grain elevator accelerate grain as the chain makes a 180° turn at the top of the elevator. The centrifugal acceleration generated as the paddles make the turn causes the grain to separate from the paddles. The grain then contacts the elevator enclosure and falls toward the base of the bin loading auger. At this location, where grain impacts the elevator housing, impact-type sensors are placed. The impact sensor measures the mass flow by sensing the force of grain as it impacts a plate, using a linear potentiometer to measure the deflection of the beams holding the plate (Shearer et al., 1999).

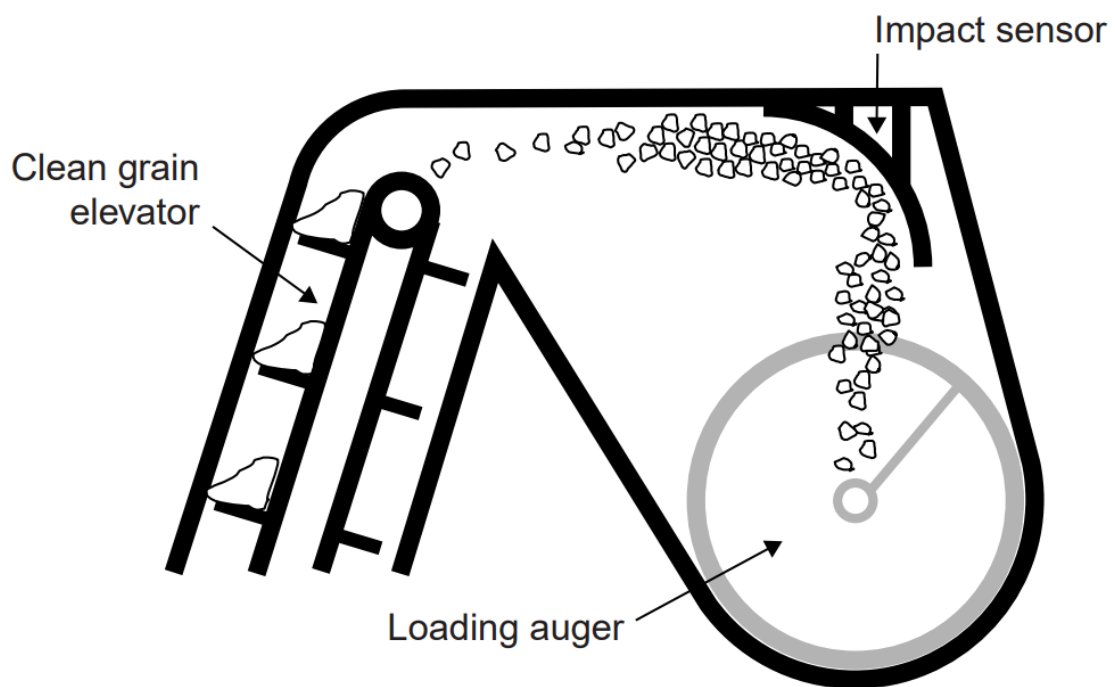


Figure 1.2: Location of the mass-flow sensor within the clean grain elevator, source: Shearer et al. (1999)

2 Methods and materials

2.1 Study area

The study areas are located in South Moravia and Ústí nad Labem (North Bohemia), the Czech Republic. Geologically, the Czech Republic belongs, for the most part, to the Bohemian Massif, characterized by hills and uplands. The most prominent soil type of this region is brown earth. However, lowland regions even retain chernozem – black-colored, highly fertile soil rich in organic matter.

2.1.1 South Moravian Region, Znojmo district

South Moravia is a region with the warmest climate in the Czech Republic. Agriculture flourishes mainly in lowlands, and the region is well-known for its wine production.

The study area comprises five wheat fields located in between the towns of Znojmo and Moravský Krumlov.

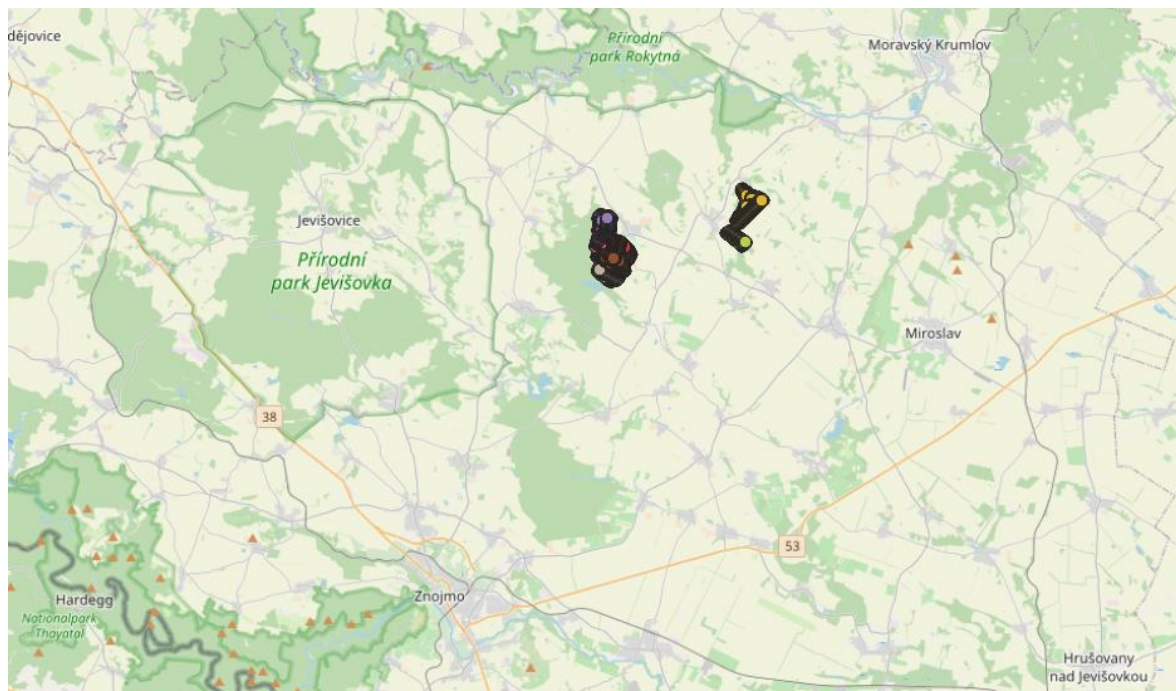


Figure 2.1: Location of study area within Znojmo district marked as colored points, source: OpenStreetMap

2.1.2 Ústí nad Labem Region, Litoměřice district

Agriculture in the region is concentrated mainly in lowlands adjacent to Labe and Ohře rivers. With its largest hop-growing sub-region Žatecko, Ústí nad Labem region dominates hop production in the Czech Republic.

The study area includes three wheat fields near the town of Roudnice nad Labem.

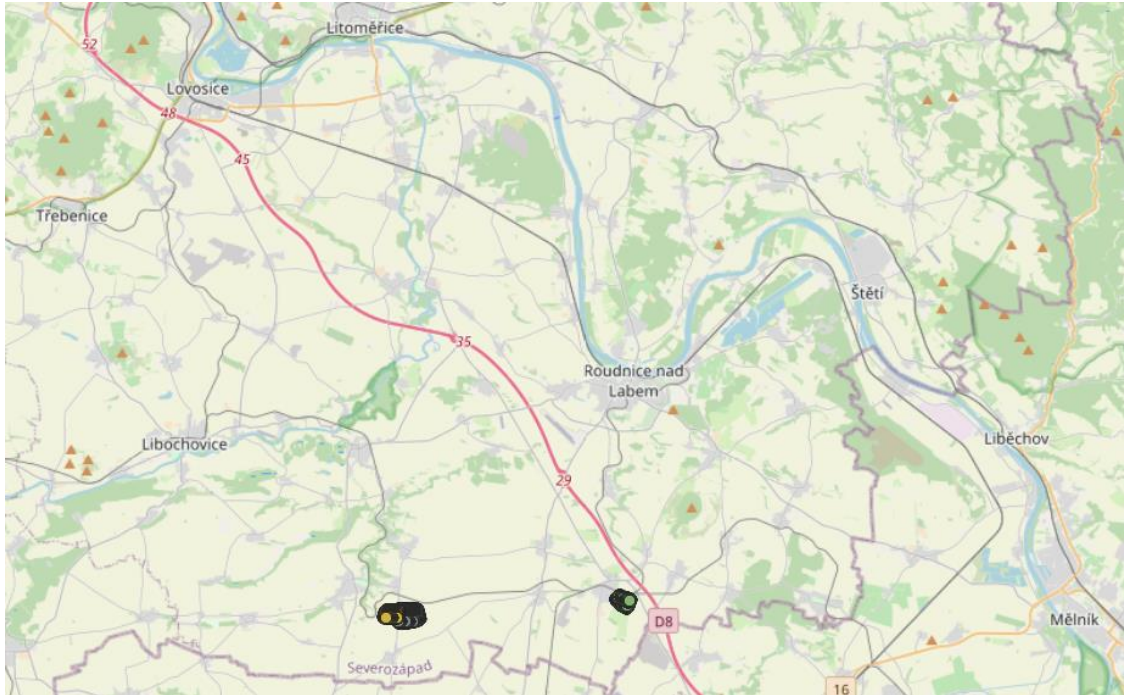


Figure 2.2: Location of the study area within Litoměřice district marked as colored points, source: OpenStreetMap

2.2 Yield data acquisition (dependent variable acquisition)

Yield data were gathered from all eight fields using grain flow meter attached to the harvester during the harvesting season of 2021. The grain flow meter measured nine variables, however, in this thesis only four of them are used. These variables were gathered on point-by-point basis, with GPS coordinates attached to each point. The essential variable is the actual crop yield in tons per hectare. Other variables taken into consideration are:

- Width of harvester swath in meters
- Moisture of gathered crop in percentages
- Elevation in meters above sea level
- Distance between current and previous point in meters

True yield is the only variable that was considered a **dependent** variable (a “target” variable, a variable that is predicted by regression models). All other variables were either treated as **independent** variables (variables used to predict the dependent/target

variable) or not included in the training process at all (see simple linear regression in the Section **Error! Reference source not found.**).

Figure 2.3 shows color-coded true crop yield in each field obtained using grain flow monitor, color-coding legends are placed alongside the plots.

It is also important to note that the four independent variables were only used as supplement independent variables in some models. The main dependent variable in all models are VI variables introduced in the next chapter.

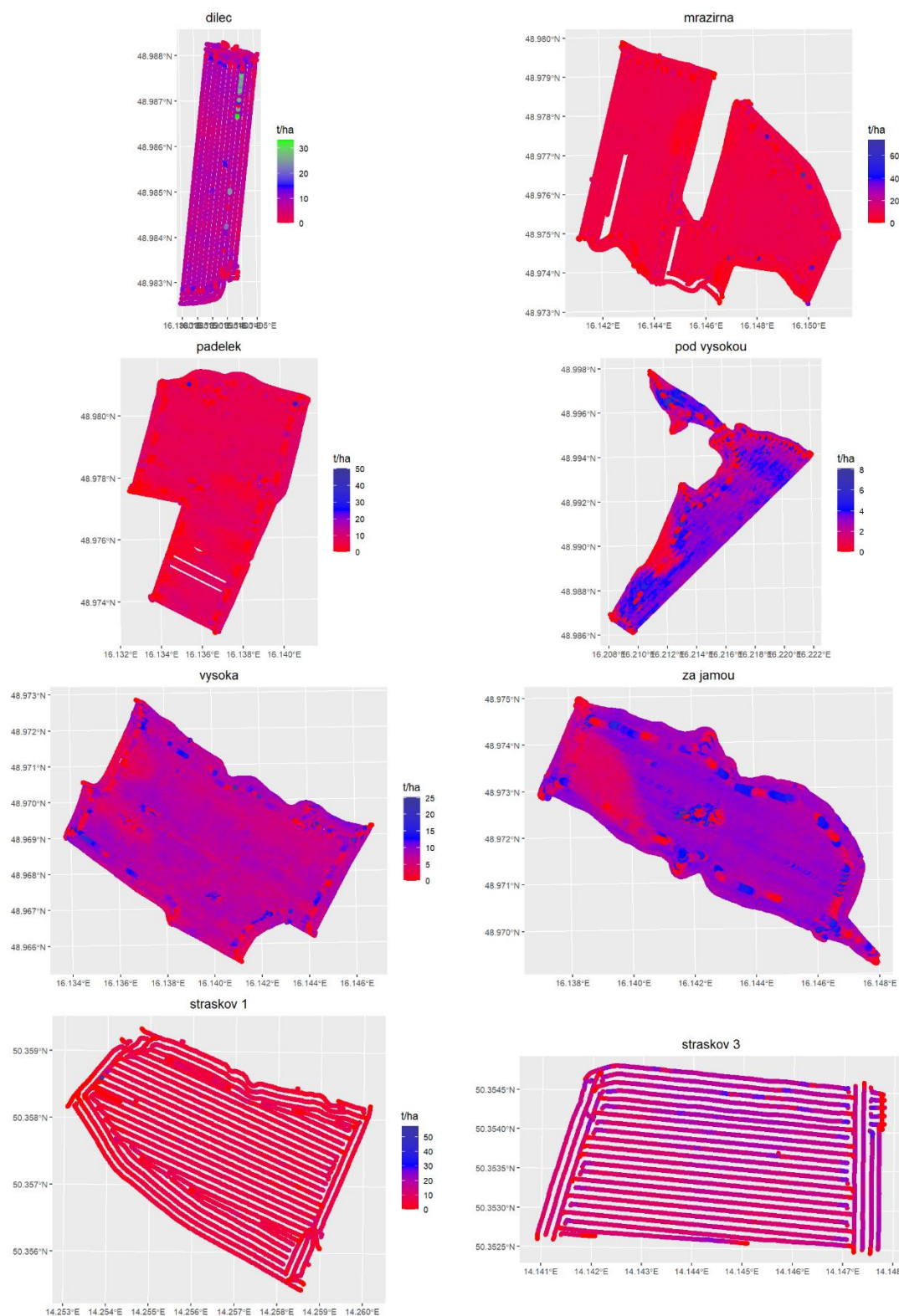


Figure 2.3: Color-coded true crop yield across all study fields (image is inserted in high-resolution, feel free to zoom in), source: author

2.3 Image data acquisition (independent variable acquisition)

In order to extract VIs (and thus obtain the independent variable for regression models), it is necessary first to acquire a multi-band aerial image of the study area. The process of VI extraction is described in Section 2.4.7.

The author chose satellite images captured by Sentinel-2 satellites as the source of VIs. These were accessed and downloaded through The Copernicus Open Access Hub, which can be found at The European Space Agency (n.d.) website. Three factors limited the image acquisition; cloud coverage, satellite revisit time, and the limited time duration of wheat crop heading and flowering stages. The heading and flowering stages were chosen as the best period to estimate wheat yield since Becker-Reshef and Franch advise so (Becker-Reshef et al., 2010; Franch et al., 2015). Satellite images of both districts were thus captured on the 26th of May 2021 since it corresponds with the mentioned stages.

2.4 Preprocessing

QGIS 3.16 (QGIS) was chosen as data preprocessing GIS software. The following procedures were done in QGIS; most of them were later automated in R language and can be found on the project's GitHub repository (vkuzdas, 2022). Some code samples can be found in Code 1, Code 2, and Code 3. For an overview of the whole process, see Figure 2.8.

2.4.1 Synchronizing coordinate reference system

The goal of preprocessing was to transform yield data and satellite images onto the same grid so that each pixel from one dataset corresponds to a pixel from the other dataset. Since yield data was captured in coordinate reference system WGS 84, it was decided to synchronize it onto the satellite's WGS 84 / UTM zone 33N .

2.4.2 Manual filtering

Two copies of yield datasets were made initially. These copies were treated separately, one copy was filtered manually, and the other one was unfiltered. The idea for such filtering was inspired by the Sudduth et al. (2012) study. Sudduth points out errors that are inherent to the nature of yield monitors attached to a harvester. The harvester usually moves in straight lines across the field, harvesting all the crops it encounters. The problem arises once the harvester reenters the already harvested line. This creates overlapping harvesting regions. Such regions are problematic because the first harvester visit gathers all the crops in the area, but the second does not, resulting in an incorrect data point with low yield since the crops were already gathered on the first visit. It is possible to notice these regions on the color-coded true crop yield map below in Figure 2.4, with errors marked in thick green circles. Overlapping regions are usually characterized by a sudden drop of yield intensity to zero. Such regions frequently occur, especially on a field of

irregular shape, forcing the harvester to merge its trails. These regions also occur on harvester machine turning points, which can be seen on the edges of fields.

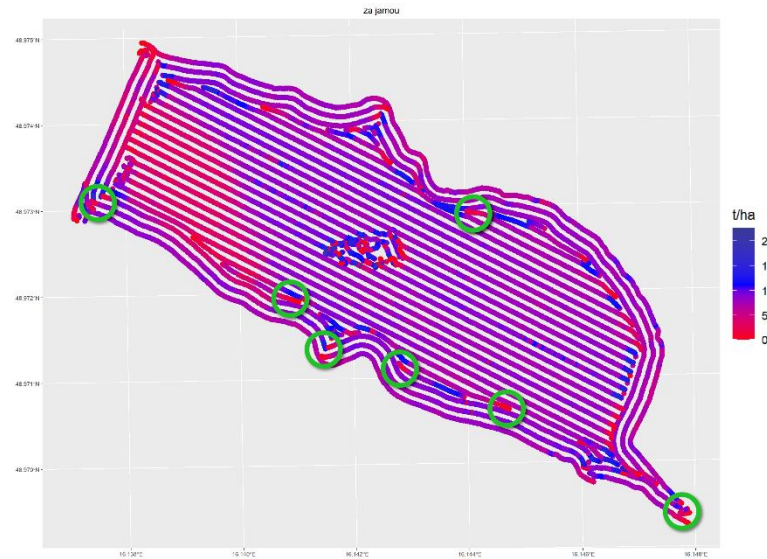


Figure 2.4: Overlapping regions – see detailed explanation in the text above, source: author

The nature of combine harvesting monitoring produces another significant error. High voltage transmission towers are placed on some arable land, making it hard to harvest the crop near it. Data produced by monitoring harvest in the proximity of transmission towers are thus error-prone. The situation is illustrated in the Figure 2.5 below.

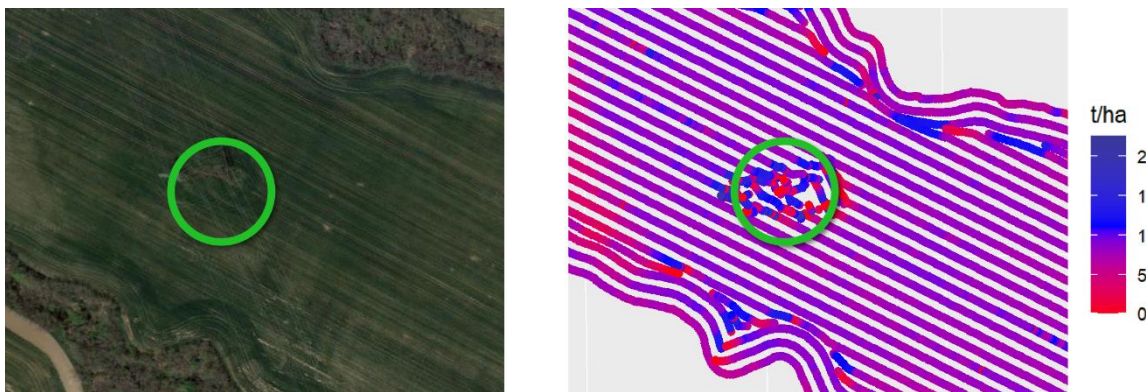


Figure 2.5: Transmission tower within field causing yield errors, source: author

To prevent the errors mentioned above, “suspicious” areas of datasets were identified and removed manually. Identification of such areas was made subjectively.

2.4.3 Percentile filtering

The author applied percentile filtering to get rid of erroneous or possible outlying observations. There was no deeper outlier analysis done beforehand.

To both manually filtered and unfiltered datasets the percentile filtering was applied. Four percentiles were selected from which upper and lower bound was calculated, the lower bound being the percentile itself. The upper bound was calculated as percentile subtracted

from 100. Values of percentiles were selected as 2.5, 5.0, 7.5, and 10.0. Each duplicate was then treated with different percentile filtering. The workflow of filtering as a whole can be seen in Figure 2.6. For the simplicity's sake, the figure displays only two percentile values.

Variables according to which the percentile filtering was done are Yield, Moisture, Swathwidth, and Distance. However, Swathwidth and Distance were filtered only from below, so only the lower bound was applied. The reasoning was that data points too close to each other are more likely to contain errors than data points that are too distant from each other.

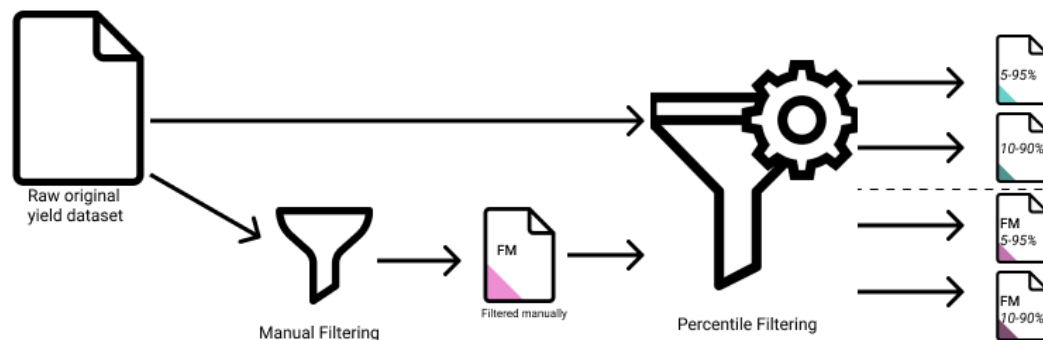


Figure 2.6: Filtering workflow diagram, source: author

2.4.4 Concave hull masking

Masking was done for several reasons. Manual filtering created “gaps” in yield maps, and these would be smoothed out by interpolating in the next step, creating false data points in the gaps as a result. Other fields had sharp edges (namely field “pod vysokou”), which would be problematic for similar reason in the next step as well. Lastly, these masks were then used to cut out satellite images so that the outline of images and yield points would be the same.

Masking was done using the QGIS tool “Concave hull”. QGIS concave hull algorithm requires an alpha argument for it to run (alpha denotes a sharpness degree of shape outline). QGIS documentation is very brief about this parameter, thus the default value of 0.3 were used in most cases. However, some fields with irregular borders required finer outlining; this was achieved using values higher than the algorithm’s default.

2.4.5 TIN interpolation

QGIS tool was also used to interpolate between yield points to create yield pixels. QGIS interpolation algorithm requires the pixel size of the output layer. Since satellite images have 10x10 meter resolution, it would makes sense to interpolate yield point data straight into 10x10m pixels. This process however results in bad positioning of the output layer. This problem is illustrated in Figure 2.7 where layer a is calculated by this approach. Layer B was instead first interpolated from points into 3x3m pixels and then reprojected and aligned by QGIS alignment tool into 10x10m pixels (see Section 2.4.6). Although Figure 2.7 shows almost identical output layers, author advises against the 10x10m interpolation

out of concern, that the output layer represents incorrect pixels after alignment onto satellite image grid. According to author's opinion, this approach achieves good computation performance while providing acceptable precision after alignment.

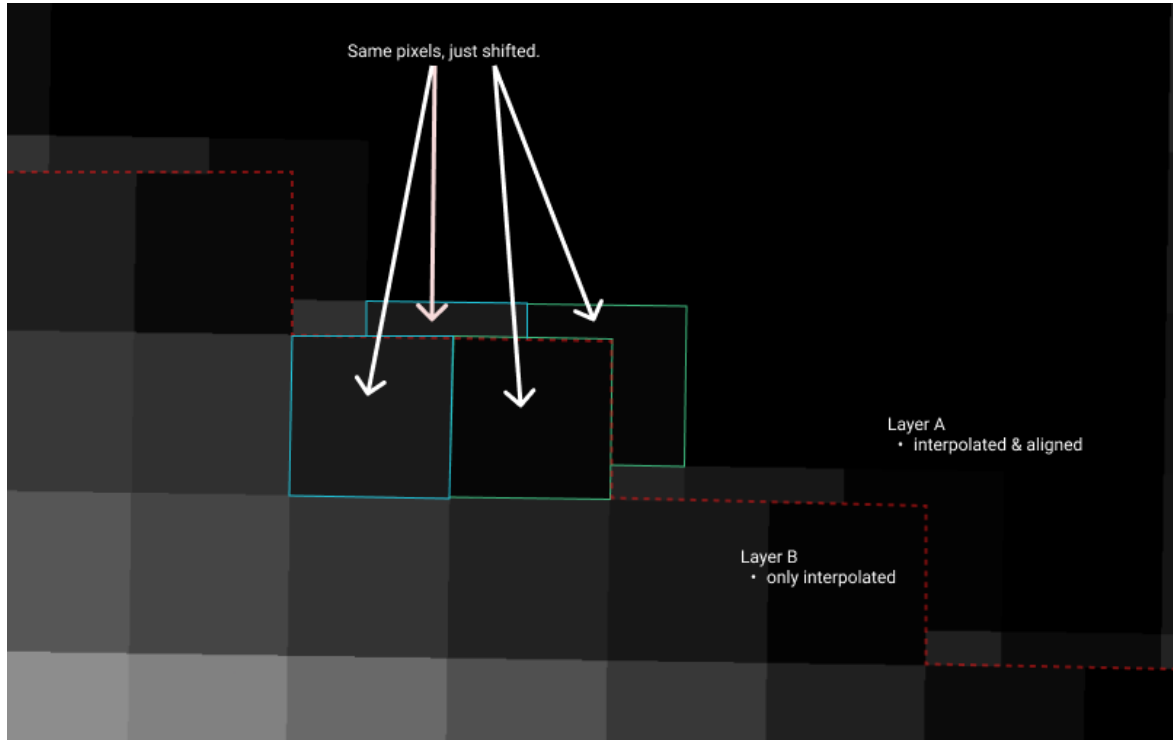


Figure 2.7: Layer B - points interpolated onto 10m pixels projected over Layer a - interpolated onto 10m and aligned onto satellite's pixels, source: author

2.4.6 Raster alignment

The final step of yield data preprocessing is to align and reproject 3m yield data onto a 10m satellite's pixel grid. This was done in QGIS with a raster alignment tool, a pixel-by-pixel alignment tool. Satellite image was selected as yield data reference layer.

2.4.7 Extracting vegetation indices

EVI and NDVI indices were extracted from satellite imagery. Indices were calculated by entering their equation into the QGIS raster calculator tool, which provides basic band arithmetics. Satellite images did not require further processing since yield data were synchronized onto them instead.

2.4.8 Exporting

Independent (vegetation indices) and dependent (true yield measured) variables, along with other harvester-measured variables (supplement independent variable), were then exported into a standard comma-separated file format (CSV). These files can be found on

project's github repository (vkuzdas, 2022). These files comprise of about 86 000 rows and 9 columns. Each files consists of roughly 8 000 rows. It is important to note, that each file only represents different filtering technique, not a unique data points, i. e., the data are stored in so-called long format. Datafiles with applied filtering are listed in Table 2-1.

Table 2-1: List of exported csv files and their respective filtering

File	Percentile	Filtering type
p_2.5.csv	2.5	Percentile only
p_5.csv	5	Percentile only
p_7.5.csv	7.5	Percentile only
p_10.csv	10	Percentile only
f_2.5.csv	2.5	Manual and percentile
f_5.csv	5	Manual and percentile
f_7.5.csv	7.5	Manual and percentile
f_10.csv	10	Manual and percentile

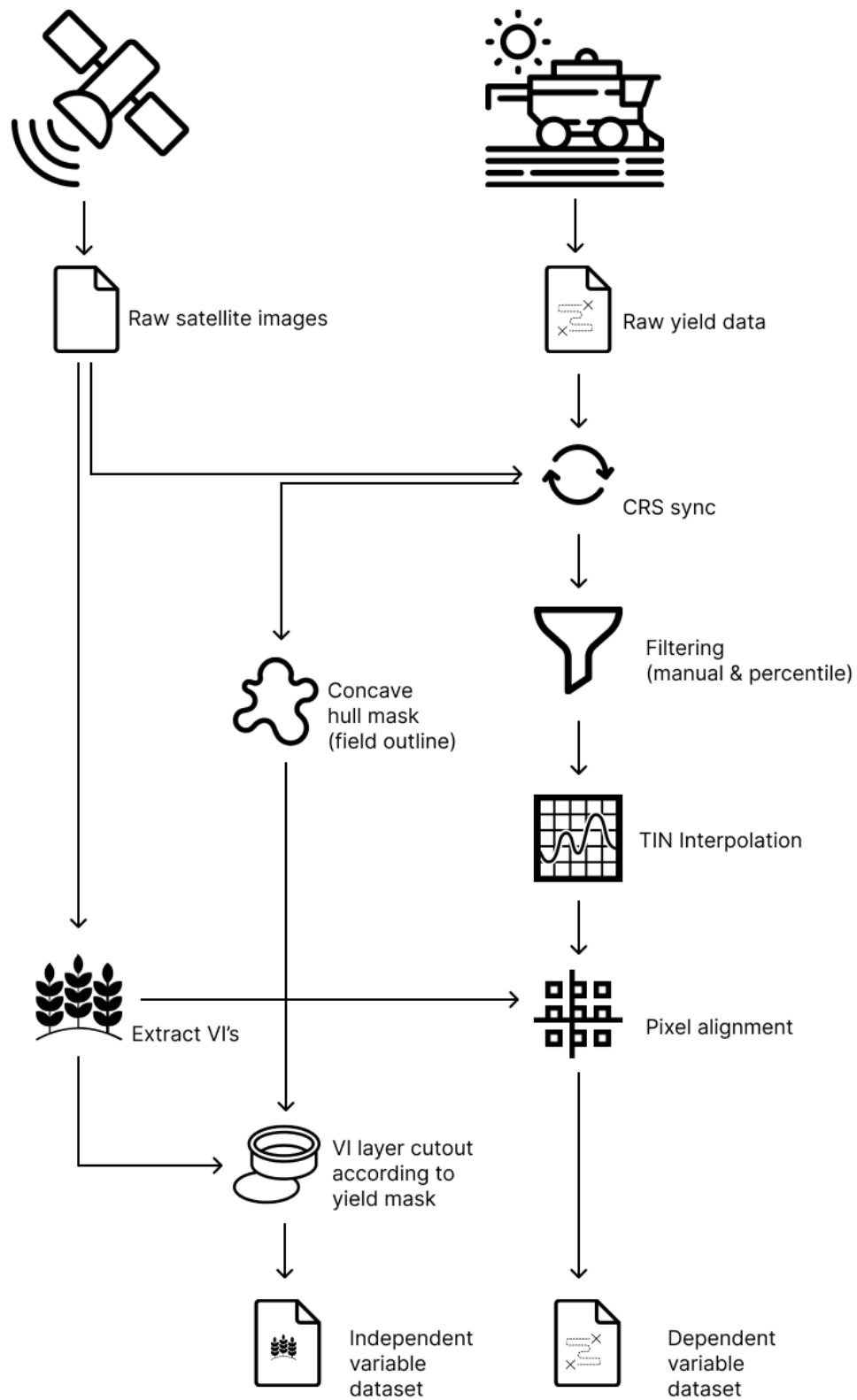


Figure 2.8: Preprocessing workflow diagram, source: author

2.4.9 Note on programmatic preprocessing with R language

Although QGIS has a fairly sophisticated user interface, it was decided to automate most of the software tasks in R language. Manual filtering is an exception of course since this requires the user's subjective evaluation of a particular field area. Interpolation, concave hull masking, and layer clipping by mask were done using *qgisprocess* library, which connects to the QGIS binaries and applies the QGIS algorithm onto selected input. GDAL library *gdalUtils* was imported in order to align raster layers. Lastly filtering was done in native R by simply calculating percentiles.

2.5 Regression analysis

Regression analysis is a statistical technique for exploring and modeling the relationship between variables. Its applications are numerous, and thus the method occurs in almost every field, including engineering, physical and chemical sciences, economics, management, and even the social sciences. It may well be the most widely used statistical technique (Montgomery et al., 2021).

Regression analysis was used as the central method for crop yield prediction in this thesis, as several previous works show its potential in this field of study (Becker-Reshef et al., 2010; Canata et al., 2021; Chlingaryan et al., 2018; Franch et al., 2015; Mehdaoui and Anane, 2020; Panda et al., 2010; Rodrigues et al., 2018; Wahab et al., 2018).

2.5.1 Linear regression

Linear regression is an excellent tool for predicting quantitative responses. It has been around for a long time, and it has been a focus of a great many textbooks. Despite its simplistic nature compared to other statistical learning approaches, linear regression is very useful and widely used. (Sohil et al., 2022)

As an example of a situation where simple linear regression may prove very useful, let us take a look at the housing market. When considering the price of a house, one may claim that it rises with the area of the house. It is possible to select a sample of houses and plot the respective variables on the x and y axes. Such a scatterplot can be seen in Figure 2.9. The scatter plot displays a random sample of 81 houses from Ames housing data (kaggle, n.d.). With no surprise, the presumption about the price-area relationship seems to be confirmed.

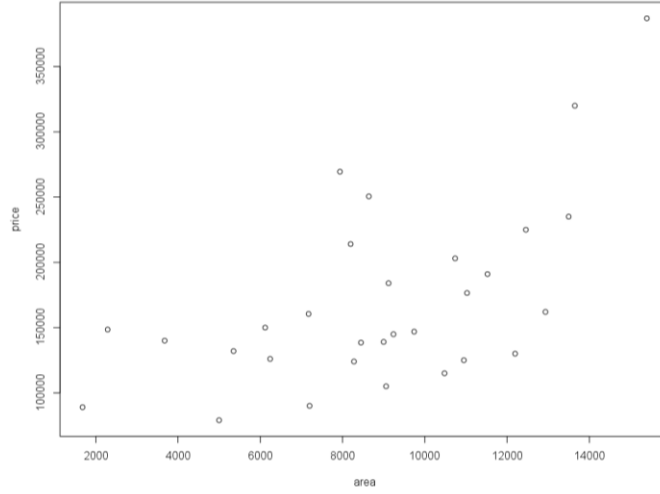


Figure 2.9: Housing market scatterplot, source: author

Further, let x represent the price and let y represent the area of the house. The equation expressing this relation is

$$y = \beta_0 + \beta_1 x, \quad (2.1)$$

where β_0 is the intercept and β_1 is the slope(Sohil et al., 2022). See Figure 2.10, displaying such plotted equation.

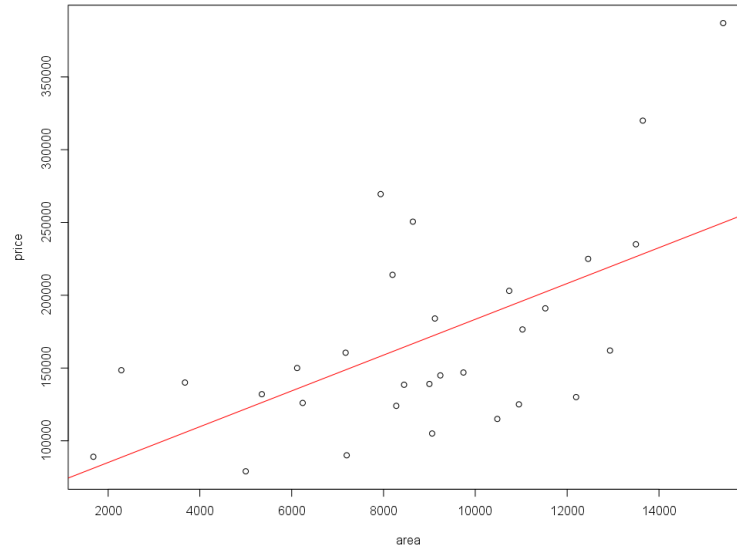


Figure 2.10: Straight-line relationship between price and area, source: author

One may argue that the data points do not follow a straight line exactly. To account for this inconvenience, let ε be the difference between the observed value of y and the straight line $\beta_0 + \beta_1 x$. Such difference is simply referred to as an error ε . It is convenient to think of the error as a random variable that accounts for the failure to fit the data exactly (Sohil et al., 2022). And so we arrive at a linear regression model expressed by the equation

$$y = \beta_0 + \beta_1 x + \varepsilon, \quad (2.2)$$

Textbooks often describe the x variable as a predictor or a regressor and the y variable as the response variable. Since the model (2.2) has only one regressor, it is called a **simple linear regression model**. In other cases, however, the response variable y may be related to more than just one regressor x . It may be related to k number of regressors, $x_1, x_2, \dots, x_k$, represented as appropriate vectors in the model. In such a case, we just scale up our model so that

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon, \quad (2.3)$$

and since there is more than one regressor, this is called a **multiple linear regression model** (Sohil et al., 2022).

Least-squares estimation of the parameters

To estimate β_0 and β_1 , the method of least squares is used. Parameters are estimated so that the sum of squared differences between each observation and the straight line is a minimum. The difference between the observed value y_i and the fitted value $\hat{y}_i$ is a **residual**. Residuals play an essential role in investigating model adequacy (Sohil et al., 2022).

In order to evaluate our model, we will turn our attention to the mentioned residuals. Given n pairs of data points (y_i, x_i) , we can express the i th residual as

$$e_i = y_i - \hat{y}_i, \quad (2.4)$$

where $i = 1, 2, \dots, n$. We can further derive a **sum of squared residuals** by the following expression

$$SS_{\text{Res}} = \sum_{i=1}^n e_i^2. \quad (2.5)$$

Last but not least, let us define the **total sum of squares** as the difference between the observed value and their mean $\bar{y}$ as

$$SS_T = \sum_{i=1}^n (y_i - \bar{y})^2. \quad (2.6)$$

With both of these sums defined, we can deduce a quantity called the **coefficient of determination** (R^2) as the ratio of SS_{Res} and SS_T subtracted from 1. This gives us a quantity in a range from 0 to 1, where “good” models have R^2 in the upper end, i.e. close to 1.0.

$$R^2 = 1 - \frac{SS_{\text{Res}}}{SS_T} \quad (2.7)$$

This quantity is often referred to as the proportion of variance explained by the regression model. R^2 is a useful measure of success in predicting the response variable (actual yield) from the regressor (Nagelkerke and Nagelkerke, 2008).

Mean squared error (MSE) is a statistic introduced to evaluate the model's predicting ability. MSE tells us how close a regression line is to a set of points. Although R^2 is a great indicator of the model's quality and accuracy, it does not answer the question of how the model performs on yet-unseen data. Since all the following models were calculated in a „training environment, “ we also want to see how these models perform in a „testing environment, “ an environment the model was not yet exposed to. By evaluating the accuracy in this new environment, we will have an idea of how well this model would perform in any other new environment. In order to evaluate this accuracy, MSE was calculated for both testing and training subsets. Ideally, we would like to select a model that has the lowest training MSE. MSE statistic can also tell us whether the model is overfitted or not since overfitting refers specifically to the case in which a model yields testing MSE smaller than training MSE(Sohil et al., 2022).

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (2.8)$$

where n stands for a number of observations in a given dataset. Inspecting the formulas for R^2 and MSE, we can see the metrics are linearly correlated, though.

2.5.2 Prediction using simple linear regression

In order to predict the actual yield based on the index value, simple regression models were calculated per dataset. All datasets were split into training and testing subsets according to a 7:3 ratio. a linear regression model was then calculated based on the training subset. Fitted models were assessed based on their coefficients and accuracy. Three statistics were used to assess the model's accuracy; R^2 , testMSE, and trainMSE.

2.5.3 Prediction using multiple linear regression

Multiple linear regression was calculated based on other variables. In contrast to simple linear regression, four additional independent variables were used. These variables were described in Section 2.2. Moreover, both indices were used as independent variables as well. The target variable was the true yield.

2.5.4 Regression trees

Let us suppose a medical center where heart attack patients are being admitted. We will measure the patient's blood pressure and ask about his age upon arrival. Based on these two variables, we can construct a graph classifying patient's overall health risks. This graph is displayed in Figure 2.11. With prior knowledge of a sample of patients, we can

determine which patients have lower (F in the graph) or higher (G in the graph) health risks.

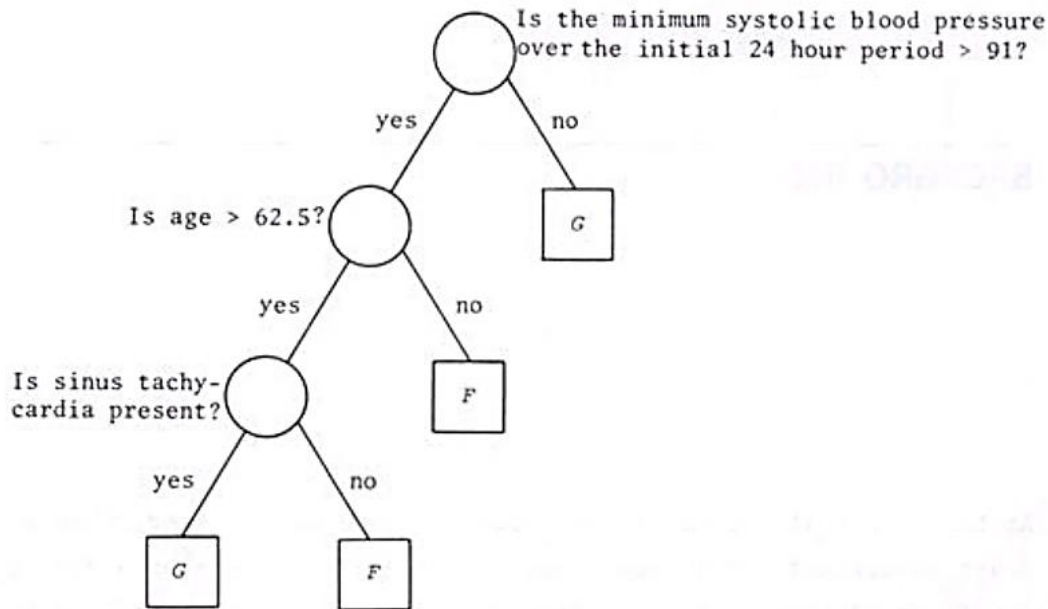


Figure 2.11: Classification of patients' health risk, source: Breiman et al. (2017)

By sketching such a graph, we have basically constructed a **decision tree**, a simple form of classifier. a classifier or a classification rule is described as a systematic way of predicting what class a given case is in. In the example above, each patient is a new case, and high/low health risk is a class the patient is classified into (Breiman et al., 2017).

A **regression tree** is very similar to the decision tree, but instead of classes, each terminal node is a constant value (Breiman et al., 2017). We can take as an example a dataset used in this thesis with the predictor variable of vegetation index NDVI and the response variable of actual crop yield in tons per hectare. Figure 2.12 displays such a tree. The tree has five constants (or five classes to use the decision tree dictionary), and the predictor space is split four times. Splitting conditions are displayed under each non-terminal node. Each node displays the average yield in tons per hectare and the percentage of cases that fall to that node.

Let us suppose a farmer is using the tree model in Figure 2.12. Just like the doctors at the medical center described above, the farmer looks up the basic variable about the incoming new case. This case is a 10x10 meter segment of a field with NDVI value of 0.20. The farmer compares the NDVI value with each split condition on each tree node. The first node states that the predicted yield of all segments with a value above 0.18 is likely to be found in the node's right child. After the second comparison in the right child, the farmer discovers that the response value is likely to be found in the left child of this node. At this moment, the farmer arrives at the terminal node, discovering that a segment with NDVI of 0.20 is likely to yield 6.1 tons per hectare on average (thus yielding 6.1 kilograms in the whole 10x10 meter segment).

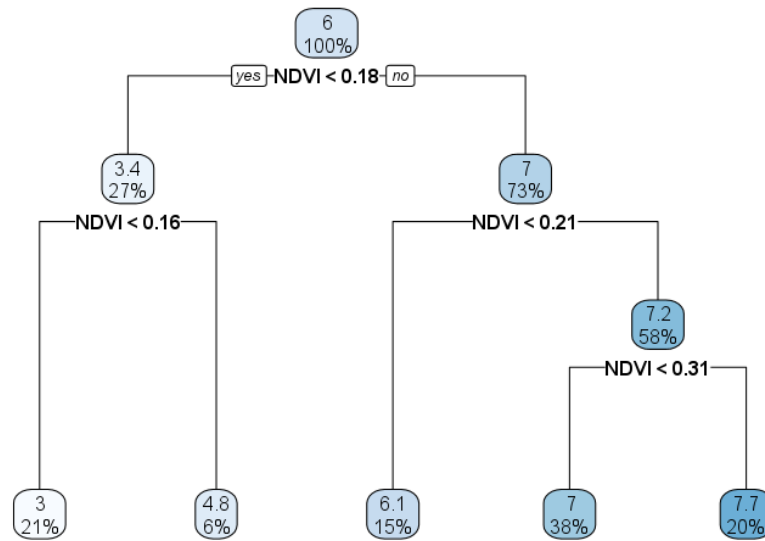


Figure 2.12: Regression tree showing a relationship between yield and NDVI, source: author

Building a Regression tree

Roughly speaking, there are two steps in the process of building a regression tree. The first step is to divide the predictor variable space (NDVI in the example above) into distinct, non-overlapping regions. In the next step, we make the same prediction for each case in the given region. This prediction is simply the mean response value of all cases within this region (Sohil et al., 2022).

To better illustrate these steps, Figure 2.13 was drawn.

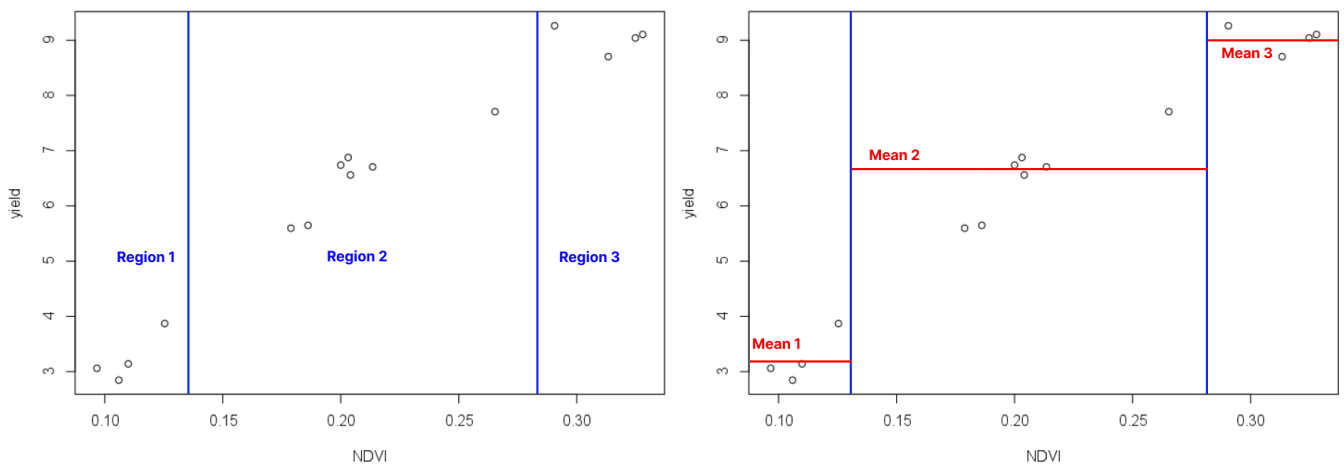


Figure 2.13: Two-step tree building process. Left: predictor values are split into non-overlapping regions. Right: Mean values of these regions are predictors for cases falling into this region. Source: author.

The process seems straightforward, but the critical question is, how to decide on splitting. In theory, regions can be of any shape. For simplicity, it is almost always decided to make splits as n -dimensional cubes depending on the number of predictors. These cubes should

ideally minimize the sum of SS_{Res} (described in Section 0) across all regions². It may, however, prove infeasible to explore all the possible combinations of cube splits. For this reason, we will take a top-down greedy³ approach called **recursive binary splitting**. We begin this process at the root node of the tree. At this point, all observations belong to a single root node's region. Then we will successively split the predictor space, branching the tree with each split. Since this approach is greedy, we will pick the best split on each split decision rather than looking forward to pick a split that will lead to a better tree in a future step (Sohil et al., 2022).

To perform the recursive binary splitting algorithm, we first have to select a predictor variable and a value of this predictor that will represent the cutpoint. Then we will split all the data into two regions—one region with values of the given predictor above the cutpoint value and one region with predictor values below the cutpoint value. Moreover, the chosen predictor-value pair must minimize the SS_{Res} . For example, if we had a dataset of ten rows with two predictors and one response variable, we would compare 20 predictor-value pairs ($2 \times 10 = 20$) and determine the pair that results in the lowest SS_{Res} . We would split the data according to the cutpoint into two regions, and then we would repeat the process in each of these regions, hence doing it recursively. This process then continues until a stopping criterion is met. For example, we may continue until no region contains more than five observations (Sohil et al., 2022).

Pruning

The algorithm described above may perform well on the training data but is likely to overfit the testing data, especially if we create many branches. At the cost of little bias, a smaller tree with fewer branches might lead to lower variance and better interpretation (Sohil et al., 2022). We can refer to Figure 2.14 for a better understanding of this trade-off.

Pruning is a strategy that aims to obtain a smaller tree (subtree) from a fairly large tree by removing some of its leaf nodes, resulting in a model that is less overfitted than the original tree. Regression tree pruning is also called the weakest link pruning since it tries to reduce the tree's complexity by removing sections that provide little predictive power (Sohil et al., 2022).

² It is helpful to imagine SS_{Res} as the distance of point from given region's mean in Figure 2.13. This distance should be minimal for each data point and its region's mean.

³ An algorithm that takes the best immediate solution while finding the answer ("greedy algorithm," n.d.).

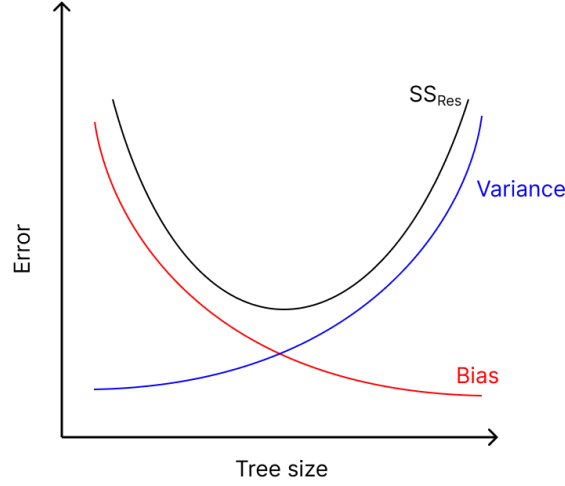


Figure 2.14: Tree model bias-variance trade-off, source: author

Yet again, the critical question is how to prune the tree to achieve good results. Ultimately, our goal with pruning is to select a subtree that leads to the lowest test rate. In order to find the lowest test rate, we will have to find a compromise between the tree's complexity and its fit to the training data. While it has been already established that these two factors can be measured effectively (complexity is the number of leaves, the fit is measured by SS_{Res}), we need to measure the compromise between the two (Sohil et al., 2022). To do this, we will introduce a “tree score” and a tuning parameter α as

$$ts = SS_{Res} + \alpha \times L, \quad (2.9)$$

where L denotes the number of leaves. The tree score is made of two parts; SS_{Res} , and a **tree complexity penalty** $\alpha \times L$, which will handicap too complex trees. We will consider a sequence of trees indexed by the parameter α . For each value of α , there will be a corresponding subtree such that the tree score is minimized (Sohil et al., 2022).

To find the best value for α , we will first build a full-sized regression tree T_0 using all data. We will prune the T_0 and increase the value of α until the tree score of pruned T_0 becomes lower than the tree score of the original T_0 . This will result in a new tree, and we will repeat this process until we end up with a sequence of trees, from a full-sized tree to just a leaf, each indexed by a corresponding α (Sohil et al., 2022).

We will split the data into a training and testing set in the next step. Next, we will use the α values we found in the previous step to build a full tree and a sequence of subtrees that minimize the tree score only from training data. Now we will calculate the SS_{Res} for each new tree using the testing data. We will repeat this process k -times to perform k -fold cross-validation, making a note of the α value that gave us, on average, the lowest SS_{Res} . This will be the final value of α (Sohil et al., 2022).

The pruning algorithm can be therefore summarized in the following steps (Sohil et al., 2022):

1. Build a large tree with a stopping criterion using all the data.

2. Out of the large tree, create a sequence of subtrees corresponding to α value which minimizes the tree score.
3. Use k -fold cross-validation to compare α values found in the previous step.
4. Select the α value that gives the best SS_{Res} on average.

Bagging

We can still do better with regard to overfitted model by using an ensemble method called **bootstrap aggregating** or **bagging** for short. An ensemble approach combines many simple models into one potentially more powerful model. Bagging is a general-purpose method for reducing the variance of a statistical learning method that is frequently being used in the context of tree models. This method is centered around the fact that averaging a set of observations reduces variance. Hence the fact is exploited by taking many training sets, building a separate prediction model for each training set, and averaging the resulting predictions. However, it is generally not possible to access multiple training sets. Thus we will bootstrap by taking repeated samples from a single training set. In other words, we will generate B different bootstrapped training data sets and train our method on the b th bootstrapped training set. After aggregating all the resulting parameters, let's say $\hat{f}^b(x)$, we will simply average them all into one single predictive model's parameter as follows,

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^b(x). \quad (2.10)$$

Constructed trees are usually grown very deep while not being pruned at all. This means that each tree has high variance and low bias (refer to Figure 2.14) (Sohil et al., 2022).

2.5.5 Prediction using single-variable regression trees

The usage of regression trees was the second method through which yield prediction was attempted. As with linear regression models, datasets were initially split into training and testing subsets according to the 7:3 ratio. Model fitting with regard to selecting predictive variables was done in a similar fashion as with linear regression; a single vegetation index was used as a predictive variable in the first place and all other variables combined in the second place. Regression tree model fitting was done using R package *rpart*.

Single-variable tree models were fitted using the default *rpart* algorithm. These are referred to as classic tree models in this thesis. After classic tree model fitting, each fitted model was examined and optimizing hyperparameters were found. These parameters were *minsplit* and *maxdepth*. Both can be used to control the training process when using *rpart*. Values for both parameters were generated into a combination grid, and a tree

model was trained based on each combination. Trained trees were sorted by cross-validation error ($xerr^4$), and the best tree was selected (lowest $xerr$). This tree is referred to as optimized.

Bagging was introduced to improve single-variable tree models even further. For this purpose, R package *caret* was used to fit such models.

2.5.6 Prediction using multi-variable regression trees

Following the single-variable regression tree fitting, multi-variable fitting was done as well. This comprised all other variables in the dataset, namely swathwidth, distance, elevation and moisture, described in Section 2.2. Same approach was taken during multi-variable tree fitting as previously during single-variable fitting; classic, optimized and bagged models were fitted.

⁴ *rpart*'s default implementation generates multiple pruned trees and calculates cross-validation error between them ("rpart.pdf," n.d.)

3 Results and Discussion

3.1 Filtering

Sudduth et al. (2012) claim that researchers have reported that 10 to 50% of the observation in a given field contain significant errors and should be removed. This claim on its own expresses how filtering yield data played a detrimental role in this study, and it very likely will play this role in similar future studies. This study, however, does not go too deep into correcting mentioned errors, and therefore its data are likely error-prone since the percentile filtering is too simple. On the other hand, this was the exact reason for introducing the manual filtering method. Although unable to be automated, an educated user may be able to correct or filter out errors that Sudduth talks about in his paper.

Nevertheless, the claim of Sudduth et al. (2012) is not just about filtering. It also sheds light on already error-prone yield data acquisition. Theoretically, the harvester operator can make significant changes to the yield data acquisition by regularly calibrating the yield monitor and optimizing the harvesting paths. However, the question is if such an ideal harvester operating is feasible during the usually hectic harvesting season.

3.2 Image acquisition

The whole process of transforming yield points onto 10x10 meter pixels is, by its nature, very flawed when it comes to error prevention. Not only does it consume computation resources, but it also erases theoretically significant sub-10x10 meter crop variance. Fortunately, this whole process can be skipped by operating an unmanned aerial vehicle (UAV) which can obtain images of much higher resolutions. This approach was recently demonstrated by Feng et al. (2020). They used a UAV with an attached RGB, multispectral, and thermal infrared camera by which they achieved a resolution between 1 to 7 cm pixels(!). The use of such a UAV could be beneficial for two reasons: it could skip the transforming process altogether, and the image acquisition would not be limited by cloud coverage and revisit time of the satellite. Moreover, a simple drone is much cheaper than a satellite.

3.3 Linear regression models

The Table 3-1 below summarizes all simple linear regression models for each datasets. Column **dataset** represents which dataset was the model trained on. These datasets have two-symbol code names separated by and underscore. The first symbol in the name denotes the filtering method: letter *p* denotes datasets upon which percentile filtering was applied exclusively and letter *f* denotes datasets which were filtered manually in the first place and then filtered additionally by percentile filtering. The second symbol denotes the

lower bound percentile according to which the percentile filtering was done. For example, a dataset f_5 denotes a dataset that was filtered manually, and then the values of dataset variables above 95th percentile and below the 5th percentile were removed (filtered out). The column **Indep. Variable** contains the index used as the independent variable in the training process. **Coefficients** column denotes the resulting equation that describes the relationship found by the linear regression model. Column **Rsq** contains the values of the coefficient of determination explained in Section 2.5.1. Lastly, columns **testMSE** and **trainMSE** contain values of the mean squared error statistic (explained in Section 2.5.1) of the test set and the train set respectively.

Going back to the overfitting criterion mentioned in Section 0 (trainMSE > testMSE), it is possible to see that simple and multiple linear regression models are mostly overfitted. This fact on its own makes these models unfortunately inconvenient.

Coefficients of simple regression indicate a positive correlation between both VI's and the true crop yield. These results correspond with findings described in Huete et al. (2002) work on VIs where higher VI values correspond with better health of the crop and, therefore, its higher yield. Similar coefficients were obtained by Mehdaoui and Anane (2020); these showed positive correlation as well.

With regard to the weaker predictive weight of the EVI index, this study confirms previous findings of Yuan et al. (2021).

Lastly, R^2 and MSE metrics seem to be positively correlated with the degree of dataset filtration. This is in accordance with initial expectations – a data with higher proportion of tail data filtered out tends to eventually contain “cleaner” values that are more closely correlated in linear fashion.

Table 3-1: Simple linear regression model assessment

Dataset	Indep. Variable	Coefficients	Rsq	testMSE	trainMSE
p_2.5	NDVI	1.74 + 18.23x	0.4759	2.5783	2.6334
	EVI	2.70 + 20.46x	0.4220	2.8367	2.9042
p_5	NDVI	1.75 + 18.17x	0.5269	2.1455	2.1380
	EVI	2.70 + 20.44x	0.4670	2.4278	2.4089
p_7.5	NDVI	1.66 + 18.80x	0.5815	1.7957	1.8165
	EVI	2.62 + 21.29x	0.5179	2.0818	2.0928
p_10	NDVI	1.56 + 19.58x	0.6335	1.5150	1.4990
	EVI	2.56 + 22.21x	0.5620	1.8105	1.7912
f_2.5	NDVI	1.95 + 18.35x	0.6144	1.6257	1.6302
	EVI	2.92 + 20.81x	0.5569	1.8596	1.8737
f_5	NDVI	1.13 + 23.02x	0.7481	1.0938	1.0609
	EVI	2.33 + 26.17x	0.6745	1.3684	1.3706
f_7.5	NDVI	1.00 + 24.11x	0.8213	0.6788	0.6951
	EVI	2.31 + 27.24x	0.7330	1.0201	1.0389
f_10	NDVI	1.04 + 24.21x	0.8219	0.6503	0.6382
	EVI	2.28 + 27.58x	0.7454	0.9116	0.9123

The Table 3-2 below lists all the multiple linear regression models trained in this thesis. These models were trained on the same datasets as the previous simple linear regression models. The main difference of these models is that each model had a total of six independent variables (NDVI, EVI, swath, elevation, distance, and moisture). Note that **Coefficients** are not listed since the equation would be very long, and the interest of this study was the models' performance rather than their equation.

The first thing to note is that almost every model achieves better R^2 values than the simple linear regression models. This can also be stated about MSE values. Moreover, the trend of higher filtration and better performance is also present in these models.

Lastly, overfitted models dominate the list again.

Table 3-2: Multiple linear regression model assessment

Dataset	Rsqr	testMSE	trainMSE
p_2.5	0.5713	2.0978	2.1542
p_5	0.6341	1.6372	1.6534
p_7.5	0.6915	1.3050	1.3393
p_10	0.7443	1.0551	1.0460
f_2.5	0.7324	1.1319	1.1313
f_5	0.8100	0.8238	0.8002
f_7.5	0.8770	0.4698	0.4787
f_10	0.8784	0.4321	0.4356

3.4 Regression tree models

Table 3-3 below shows all the classic and optimized trees defined in Section 2.5.2. The table is very similar to the two tables listed above. The only difference is columns **xerr** and **opt xerr** which refer to the cross-validation errors of classic and optimized tree models.

As well as with linear regression models, it is possible to see a lot of overfitted models with smaller testMSE than trainMSE. However, it is worth mentioning that a heavily filtered dataset *f_10* did not produce a single overfitted regression tree model. Note that manually filtered datasets yield fewer overfitted models than their manually unfiltered counterparts.

Constructing optimized trees did not produce any significant results. These models had only a marginal decrease of cross-validation error.

Table 3-3: Classic and optimized tree models

Dataset	Indep. Variable	Rsqr	xerr	opt xerr	testMSE	trainMSE
p_2.5	NDVI	0.57995	0.42396	0.42652	2.10588	2.11052
	EVI	0.51288	0.48736	0.48749	2.43110	2.44754
p_5	NDVI	0.64202	0.36111	0.36065	1.59165	1.61780
	EVI	0.56635	0.43540	0.43549	1.91929	1.95975
p_7.5	NDVI	0.70588	0.30023	0.29988	1.28248	1.27669
	EVI	0.63956	0.36623	0.36930	1.59216	1.56457
p_10	NDVI	0.75284	0.24916	0.25063	1.01328	1.01087
	EVI	0.67642	0.32594	0.32665	1.32778	1.32343
f_2.5	NDVI	0.72958	0.27467	0.27446	1.17053	1.14337
	EVI	0.65150	0.35432	0.35311	1.45658	1.47349
f_5	NDVI	0.80339	0.20314	0.20362	0.88325	0.82791
	EVI	0.72217	0.28854	0.28645	1.17927	1.16994
f_7.5	NDVI	0.85443	0.15581	0.15443	0.58632	0.56631
	EVI	0.76267	0.24034	0.23928	0.90977	0.92328
f_10	NDVI	0.87611	0.12834	0.12548	0.46723	0.44398
	EVI	0.79664	0.21196	0.21220	0.76307	0.72876

Table 3-4 lists all the regression tree models that were constructed using the bagging method. Recall that bagging was introduced with the aim of reducing the “overfittedness” of tree models. The overfitted trees stayed overfitted, however. Therefore the use of bagging was not impactful in this instance.

Otherwise, models listed in the table achieve slightly better performance overall than non-bagged models.

Table 3-4: Single-variable bagged tree models

Dataset	Indep. Variable	Rsqr	testMSE	trainMSE
p_2.5	NDVI	0.58010	2.07626	2.09033
	EVI	0.51843	2.41447	2.43220
p_5	NDVI	0.64292	1.56507	1.60318
	EVI	0.57246	1.89031	1.92833
p_7.5	NDVI	0.70807	1.25894	1.25769
	EVI	0.63927	1.58785	1.55489
p_10	NDVI	0.75720	0.98456	0.98456
	EVI	0.67857	1.30241	1.30373
f_2.5	NDVI	0.73104	1.15059	1.12478
	EVI	0.65777	1.4335	1.43926
f_5	NDVI	0.80537	0.84808	0.80342
	EVI	0.72354	1.16320	1.15323
f_7.5	NDVI	0.86368	0.50625	0.51019
	EVI	0.76420	0.88512	0.90937
f_10	NDVI	0.88102	0.43903	0.41901
	EVI	0.79486	0.76712	0.73640

Table 3-5 summarizes all classic and optimized multi-variable tree models. Just like multiple regression models, these models were trained to predict the single target variable – true crop yield, given six independent variables – NDVI, EVI, swath, distance, moisture, and elevation. These tree models have also improved by introducing more independent variables concerning the performance.

More importantly, listed models are less overfitted than previously mentioned models. Whereas previously only models trained on heavily filtered datasets were not overfitted, the table shows it is not the case in this instance. Dataset *p_2.5*, which is the most unfiltered dataset, is not overfitted.

Table 3-5: Classic and optimized multi-variable tree models

Dataset	Rsqr	xerr	opt xerr	testMSE	trainMSE
p_2.5	0.65076	0.35094	0.35434	1.82452	1.75474
p_5	0.72222	0.28912	0.28989	1.24930	1.25536
p_7.5	0.76363	0.23908	0.23947	1.03916	1.02600
p_10	0.80981	0.19441	0.19314	0.77280	0.77785
f_2.5	0.84015	0.17081	0.17143	0.71444	0.67588
f_5	0.87486	0.13424	0.13407	0.58886	0.52696
f_7.5	0.89915	0.10429	0.10411	0.39001	0.39232
f_10	0.90732	0.09459	0.09394	0.35689	0.33213

Lastly Table 3-6 lists all the multi-variable bagged regression tree models.

Although achieving slightly better performance overall, bagging surprisingly did not improve overfitted models.

Table 3-6: Multi-variable bagged regression tree models

Dataset	Rsqr	testMSE	trainMSE
p_2.5	0.67605	1.61059	1.61801
p_5	0.74336	1.19954	1.15739
p_7.5	0.78940	0.91491	0.91348
p_10	0.82474	0.69570	0.70756
f_2.5	0.84061	0.67970	0.66939
f_5	0.87093	0.56320	0.52953
f_7.5	0.91264	0.34438	0.33593
f_10	0.92747	0.26819	0.25661

3.5 Best models

The best model regarding how accurate and overfitted it is is the multi-variable bagged regression tree constructed on the *f_10* dataset in the table Table 3-6. There can be a lot of objections raised against it, however. First, this model is heavily filtered; therefore, it is

very limited in its predictive ability by not covering entire fields. Second, it is impossible to use it before harvesting since it uses variables obtained after harvesting (moisture, elevation, etc.).

To answer the question which model yields the best predictions and can be used before harvest, attention should be turned to models that are exclusively trained on indices. This model is also a regression tree model based on NDVI index from f_{10} dataset contained in Table 3-4.

Lastly, an honorable mention goes to classic multi-variable tree model from Table 3-5 trained on $p_{2.5}$ dataset – a dataset that is the least filtered, and thus contains the most datapoints. This model proved to perform fairly well while not being overfitted.

Conclusions

History is full of far more idiotic miscalculations.

- Yuval Noah Harari

While being far from perfect, some models did achieve significant performance when it comes to precision evaluated by the R^2 statistic. However, the applicability of these models is questionable since many proved to be overfitted even when overfitting precautions were taken (e.g. bagging) to prevent it.

As expected, the introduction of more independent variables into linear modeling, as well as appropriate pruning of tree models and their aggregating using bagging, helped to improve predictive performance, and even more, reduced the overfitting “curse” a bit.

To the author’s surprise, a significant portion of the research had to be devoted to data acquisition and processing, which turned out not to be trivial. However, the R language proved to be an effective tool for data processing. The other portion devoted to training could be explored in more depth in order to design better models according to the author’s opinion.

Further research in this endeavor should, in author’s opinion, pay more attention to filtering and outlier analysis. a particularly interesting source regarding filtering is Sudduth et al. (2012) and their Yield Editor 2.0 software. Other recommendations can be made regarding image acquisition which would be better with higher resolution than the 10x10m demonstrated in this thesis. Higher resolution can be easily, acquired by UAVs, for example. Further research should also explore other models, not only linear and tree regression.

List of references

1. Becker-Reshef, I., Vermote, E., Lindeman, M., Justice, C., 2010. A generalized regression-based model for forecasting winter wheat yields in Kansas and Ukraine using MODIS data. *Remote Sens. Environ.* 114, 1312–1323. <https://doi.org/10.1016/j.rse.2010.01.010>
2. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J., 2017. *Classification And Regression Trees*. Routledge, New York. <https://doi.org/10.1201/9781315139470>
3. Canata, T.F., Wei, M.C.F., Maldaner, L.F., Molin, J.P., 2021. Sugarcane Yield Mapping Using High-Resolution Imagery Data and Machine Learning Technique. *Remote Sens.* 13, 232. <https://doi.org/10.3390/rs13020232>
4. Chlingaryan, A., Sukkarieh, S., Whelan, B., 2018. Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review. *Comput. Electron. Agric.* 151, 61–69. <https://doi.org/10.1016/j.compag.2018.05.012>
5. FAO (Ed.), 2017. *The future of food and agriculture: trends and challenges*. Food and Agriculture Organization of the United Nations, Rome.
6. Feng, A., Zhou, J., Vories, E.D., Sudduth, K.A., Zhang, M., 2020. Yield estimation in cotton using UAV-based multi-sensor imagery. *Biosyst. Eng.* 193, 101–114. <https://doi.org/10.1016/j.biosystemseng.2020.02.014>
7. Franch, B., Vermote, E., Becker-Reshef, I., Claverie, M., Huang, J., Zhang, J., Justice, C., Sobrino, J., 2015. Improving the timeliness of winter wheat production forecast in the United States of America, Ukraine and China using MODIS data and NCAR Growing Degree Day information. *Remote Sens. Environ.* 161. <https://doi.org/10.1016/j.rse.2015.02.014>
8. Gessesse, A.A., Melesse, A.M., 2019. Temporal relationships between time series CHIRPS-rainfall estimation and eMODIS-NDVI satellite images in Amhara Region, Ethiopia, in: *Extreme Hydrology and Climate Variability*. Elsevier, pp. 81–92. <https://doi.org/10.1016/B978-0-12-815998-9.00008-7>
9. Goel, P.K., Prasher, S.O., Landry, J.A., Patel, R.M., Bonnell, R.B., Viau, A.A., Miller, J.R., 2003. Potential of airborne hyperspectral remote sensing to detect nitrogen deficiency and weed infestation in corn. *Comput. Electron. Agric.* 38, 99–124. [https://doi.org/10.1016/S0168-1699\(02\)00138-2](https://doi.org/10.1016/S0168-1699(02)00138-2)
10. greedy algorithm [WWW Document], n.d. URL <https://xlinux.nist.gov/dads//HTML/greedyalgo.html> (accessed 4.12.22).
11. Huete, A., Didan, K., Miura, T., Rodriguez, E.P., Gao, X., Ferreira, L.G., 2002. Overview of the radiometric and biophysical performance of the MODIS vegetation indices. *Remote Sens. Environ.* 83, 195–213. [https://doi.org/10.1016/S0034-4257\(02\)00096-2](https://doi.org/10.1016/S0034-4257(02)00096-2)

12. kaggle, n.d. House Prices - Advanced Regression Techniques [WWW Document]. URL <https://kaggle.com/competitions/house-prices-advanced-regression-techniques> (accessed 5.8.22).
13. Kenneth A Sudduth, Scott T Drummond, D Brenton Myers, 2012. Yield Editor 2.0: Software for Automated Removal of Yield Map Errors, in: 2012 Dallas, Texas, July 29 - August 1, 2012. Presented at the 2012 Dallas, Texas, July 29 - August 1, 2012, American Society of Agricultural and Biological Engineers. <https://doi.org/10.13031/2013.41893>
14. Lal, R., 2004. Soil carbon sequestration impacts on global climate change and food security. *Science* 304, 1623–1627. <https://doi.org/10.1126/science.1097396>
15. McBratney, A., Whelan, B., Ancev, T., Bouma, J., 2005. Future Directions of Precision Agriculture. *Precis. Agric.* 6, 7–23. <https://doi.org/10.1007/s11119-005-0681-8>
16. Mehdaoui, R., Anane, M., 2020. Exploitation of the red-edge bands of Sentinel 2 to improve the estimation of durum wheat yield in Grombalia region (Northeastern Tunisia). *Int. J. Remote Sens.* 41, 8986–9008. <https://doi.org/10.1080/01431161.2020.1797217>
17. Nagelkerke, N.J.D., Nagelkerke, N.J.D., 2008. A note on a general definition of the coefficient of determination.
18. Panda, S.S., Ames, D.P., Panigrahi, S., 2010. Application of Vegetation Indices for Agricultural Crop Yield Prediction Using Neural Network Techniques. *Remote Sens.* 2, 673–696. <https://doi.org/10.3390/rs2030673>
19. Rodrigues, F.A., Blasch, G., BlasDefournych, P., Ortiz-Monasterio, J.I., Schulthess, U., Zarco-Tejada, P.J., Taylor, J.A., Gérard, B., 2018. Multi-Temporal and Spectral Analysis of High-Resolution Hyperspectral Airborne Imagery for Precision Agriculture: Assessment of Wheat Grain Yield and Grain Protein Content. *Remote Sens.* 10, 930. <https://doi.org/10.3390/rs10060930>
20. rpart.pdf, n.d. URL <https://cran.r-project.org/web/packages/rpart/rpart.pdf>
21. Segarra, J., Buchailot, M.L., Araus, J.L., Kefauver, S.C., 2020. Remote Sensing for Precision Agriculture: Sentinel-2 Improved Features and Applications. *Agronomy* 10, 641. <https://doi.org/10.3390/agronomy10050641>
22. Shearer, S.A., Fulton, J.P., McNeill, S.G., Higgins, S.F., Mueller, T.G., 1999. Pa-1: Elements of Precision Agriculture: Basics of Yield Monitor Installation and Operation 10.
23. Sohil, F., Sohali, M.U., Shabbir, J., 2022. An introduction to statistical learning with applications in R: by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, New York, Springer Science and Business Media, 2013, \$41.98, eISBN: 978-1-4614-7137-7. *Stat. Theory Relat. Fields* 6, 87–87. <https://doi.org/10.1080/24754269.2021.1980261>
24. Stehfest, E., Bouwman, L., van Vuuren, D.P., den Elzen, M.G.J., Eickhout, B., Kabat, P., 2009. Climate benefits of changing diet. *Clim. Change* 95, 83–102. <https://doi.org/10.1007/s10584-008-9534-6>

25. The European Space Agency, n.d. URL <https://scihub.copernicus.eu/dhus/#/home> (accessed 5.8.22).
26. vkuzdas, 2022. URL https://github.com/vkuzdas/yield_prediction (accessed 5.8.22).
27. Wahab, I., Hall, O., Jirström, M., 2018. Remote Sensing of Yields: Application of UAV Imagery-Derived NDVI for Estimating Maize Vigor and Yields in Complex Farming Systems in Sub-Saharan Africa. *Drones* 2, 28. <https://doi.org/10.3390/drones2030028>
28. Yuan, N., Gong, Y., Fang, S., Liu, Y., Duan, B., Yang, K., Wu, X., Zhu, R., 2021. UAV Remote Sensing Estimation of Rice Yield Based on Adaptive Spectral Endmembers and Bilinear Mixing Model. *Remote Sens.* 13, 2190. <https://doi.org/10.3390/rs13112190>

Annexes

Annex A: Source code samples

Code 1: Main preprocessing function

```
1 preprocess <- function(points, flt_quant, itp_attribute) {
2   # 1) filter each .shp
3   filtered_point_files <- lapply(
4     X = points,
5     FUN = filter_all_point_attributes,
6     quant = flt_quant
7   )
8
9   # 2) interpolate each .shp
10  yield_points_files_interpolated <- lapply(
11    X = filtered_point_files,
12    FUN = interpolate_point_layer,
13    itp_attribute = itp_attribute
14  )
15
16  # 3) convert interpolated onto 10m
17  # use the big indices_p
18  # big indices_p -> big overall pic
19  aligned <- lapply(
20    X = yield_points_files_interpolated,
21    FUN = align_layer
22  )
23  # 3b) masks
24  alpha_per_file <- c(
25    0.03, 0.03, 0.05, 0.03, 0.03, 0.03, 0.1, 0.1, 0.03
26  )
27  files_and_alphas <- data.frame(
28    layer = unlist(filtered_point_files),
29    alpha = alpha_per_file
30  )
31  masks <- apply(
32    files_and_alphas,
33    MARGIN = 1,
34    function(x) create_mask_from_points(x[1], x[2])
35  )
36
37  # 4) cut interpolated 10m layers
38  df_to_clip <- data.frame(layer = unlist(aligned), mask = masks)
39  interpolated_clipped <- apply(
40    X = df_to_clip,
41    MARGIN = 1,
42    FUN = function(x) clip_interpolated(x[1], x[2])
43  )
44
45  # 5) cut 10m indices_p
46  clipped_EVIs <- lapply(X = masks, FUN = get_clipped_EVI)
47  clipped_NDVis <- lapply(X = masks, FUN = get_clipped_NDVI)
48 }
```

Code 2: Function for linear model construction

```
1 assess <- function(path, index) {
2   stopifnot(index == "NDVI" || index == "EVI")
3   writeLines("")
4   writeLines(path)
5   writeLines(paste0("YIELD ~ ", index))
6   data <- read.csv(path)
7
8   set.seed(123)
9   split <- initial_split(data, prop = .7)
10  train <- training(split)
11  test <- testing(split)
12
13  if(index == "NDVI") {
14    model <- lm(YIELD ~ NDVI, data = train)
15  } else {
16    model <- lm(YIELD ~ EVI, data = train)
17  }
18
19  write_lm_assessment(model, test, train, 4)
20 }
```

Code 3: Function for linear model assesment

```
1 write_lm_assessment <-function(model, test_set, train_set, round) {
2
3   # equation
4   eq <- tidy(model)
5   writeLines(
6     paste0(
7       round(eq$estimate[1],2), " + ",
8       round(eq$estimate[2],2), "x"
9     )
10  )
11
12  # RSE
13  rse <- sigma(model)
14  writeLines(paste0("RSE = ", round(rse,4)))
15
16  # percentage error
17  err <- sigma(model)/mean(train_set$YIELD)
18  writeLines(paste0("%err = ", round(err,4)))
19
20  r2 <- rsquare(model, data = train_set)
21  writeLines(paste0("R^2 = ", round(r2,4)))
22
23  testMSE <- test_set %>%
24    add_predictions(model) %>%
25    summarise(MSE = mean((YIELD - pred)^2))
26  writeLines(paste0("testMSE = ", round(testMSE,4)))
27
28  trainMSE <-train_set %>%
29    add_predictions(model) %>%
30    summarise(MSE = mean((YIELD - pred)^2))
31  writeLines(paste0("trainMSE = ", round(trainMSE,4)))
32 }
```